Quadrats and inner/outer coordinates

Gabriel Arellano

2026-08-30

Contents

1 Introduction

1.1 Quadrats and coordinates

In forest data, large sampling units (e.g. plots) are often split into smaller units (e.g. subplots, quadrats, or sub-quadrats of different sizes). One important reason is fieldwork: data are recorded at the quadrat level, with quadrat-level coordinates, and then aggregated at the plot level, with plot-level coordinates. Other important pieces of information can be stored at the quadrat level, e.g. habitat "maps" as tables of the form {quadrat ID, habitat}.

Because of the multiple spatial frameworks, we can find multiple variables for x and y coordinates:

- within the whole plot (e.g. "global" or plot coordinates in ForestGEO, {gx, gy} or {PX, PY})
- within subplots or quadrats (e.g. "local" or 20 x 20 m quadrat coordinates in ForestGEO, {lx, ly} or {QX, QY})
- within smaller units (e.g. even more local coordinates within 5 x 5 m sub-quadrats)

The vocabulary chosen here is slightly more general:

- outer coordinates: relative to whatever sampling unit that can be subdivided into smaller sub-units.
- inner coordinates: relative to the smaller sub-units in which a larger sampling unit is split.
- quadrat, for the sub-units in which a larger unit (plot, subplot, etc.) is split. It does not refer to any spatial scale in particular. Here, "quadrat" could mean half of the plot, or one hectare, or one 1x1 m quadrat within a 5x5 m quadrat within a 20 x 20 m quadrat, etc.

Because the approach is general, just pieces within other pieces, the outer coordinates may refer to a whole 50-ha plot, or only within a specific hectare within it, depending on the context. Similarly, the corners of small 5 x 5 m quadrats could refer to their location within the entire plot, or their (nested) location within specific 20 x 20 m quadrats. The code developed here does things at *any* scale, and does *not* track coordinates that refer to an absolute or whole-plot-level scale, beyond of whatever the user inputs. If the user wants to track whole-plot-level coordinates, then they simply have to use within-whole-plot coordinates consistently to express quadrat-level locations, for any level of nestedness or resolution to which those quadrats may refer.

1.2 Good vs. bad practices

If the project has quadrat-level data, there is a need for quadrat-level IDs and at least one quadrat-level table. The most important information in such quadrat-level table is quadrat location. The link with other tables is made as with any other table: through shared keys = quadrat-level ID. Without any extra assumptions.

Storing location implicit in a quadrat-level ID is a bad practice. If quadrat location is implicit in quadrat ID, one cannot specify {quadrat IDs, locations} without changing their codes or indexes. This is a great point of vulnerability in data management, including CTFS scripts and `fgeo` R package.

If, for whatever reason, a team / project / network does not store quadrat locations explicitly, then they must be inferred from the quadrat name or ID. This step of finding location from ID must be rare, separated, explicit. It is just a hack, and it is a risky operation in general. It should be done as soon as possible and checked manually with care. It is not something that should be done again and again within analytical pipelines or by default within other functions.

In short: for a given team or network, a general recommendation is: get quadrat-level {xmin, ymin, xmax, ymax} coordinates as soon as possible, store them, and use best practices from then on. You would relate with quadrat-level info the same way you relate with species-level info, habitat-level info, etc: through explicit unique IDs and explicit information stored in tables, not conventions.

If the project has quadrat-level data at different scales (e.g. at the hectare level, at the 20x20m scale, at the 5x5 m scale, etc.) then these can be treated simply as different types of observations. Their location relative to each other should be immediate to derive from their locations, as happens with the relative locations of any other type of spatial objects: quadrats within hectares, or sub-quadrats within sub-plots is not different than trees within habitats, habitats within plots, or trees within quadrats. It does not require additional conventions beyond their spatial location.

As said above, the code developed here does things at *any* scale, and does *not* track coordinates that refer to an absolute or whole-plot-level scale, beyond of whatever the user inputs. The way to keep track of things relative to a whole-plot-level scale is to always express quadrat-level locations relative to the whole plot, even if the quadrats come from a nested structure. Using coordinates that refer to the whole plot is also the most obvious way in which quadrat-level locations, at any resolution, can be stored permanently.

2 Functions

The new code targets the same general tasks that CTFS and `fgeo` code did. Most importantly, it goes from outer (global) to inner (local) coordinates and *vice versa*. This new version of

the code is articulated around the idea that the user has to arrive, as soon as possible, to the quadrat-level locations. Then those locations must be stored permanently and used from then on, following good practices. The new code also tries to fulfill other general criteria, like reducing the number of functions and dependencies, and make the functions more usable outside "orthodox ForestGEO" tables.

Two functions create quadrats from scratch:

- `make_quadrats_fast()`: This is the primitive; it generates quadrats of a given resolution, with simple IDs. It is auxiliary for other functions, but also the direct way of splitting a point dataset into quadrats of varying resolution.
- `make_quadrats()`: This is more general. It generates quadrats of a given size, in ways congruent with a specific quadrat ID convention. The output is a quadrat-location table with one row per quadrat and columns `quadrat_id`, `xmin`, `ymin`, `xmax`, and `ymax`. The coordinate columns describe each quadrat in the outer coordinate frame. You would have to describe the ID convention through a long set of descriptive parameters (see devoted section below). This function is necessary because some teams have a specific quadrat naming or IDing convention (or two or more conventions), but not the quadrat locations.

An additional function helps to obtain the quadrat location table from a dataset with tree locations:

- `get_quadrat_locations_from_outer_coordinates()`: Use this if you have a set of observations already grouped into quadrats (e.g. {tree id, quadrat id, tree x, tree y}). The function will infer the characteristics of the grid and return the location of the quadrats as a table with one row per quadrat, of the form {quadrat ID, xmin, ymin, xmax, ymax}.

The previous functions (`make_quadrats()` and `get_quadrat_locations_from_outer_coordinates()`) are needed to get the quadrat-level coordinates, in case the team does not have them already stored in a quadrat-level table. That table should be manually checked, and after human validation, kept carefully as raw data.

Other two functions move point data (e.g. tree-level coordinates) between outer coordinates (e.g. plot-level {gx, gy}) and inner coordinates (e.g. quadrat-level {lx, ly}). Both use quadrat-level locations as input, so you must have obtained and stored those first.

- `assign_points_to_quadrats()`: Assigns points, given by outer coordinates, to quadrats, following a table of explicit quadrat locations. The output keeps the original point index, outer coordinates, assigned quadrat ID, and inner coordinates within each assigned quadrat.

  - **Warning**: this function is very flexible, and will return membership and inner coordinates into *any* set of rectangular shapes defined by the input, including standard quadrats (forming a regular, complete grid) but also overlapping quadrats, randomly located quadrats of different sizes, quadrats covering part of the plot, or outside the plot, etc. The function returns a table for each [point x quadrat] combination, so alignment with the input is not guaranteed, unless the quadrats represent a complete, regular grid, with non-overlapping quadrats. If you are using the function for flexible cases, do due diligence.

- `compile_points_from_quadrats()`: This is the reverse step: it gets inner (local) coordinates of points within specific quadrats, plus quadrat-level locations, and returns the coordinates of the points expressed within the (outer) framework: the framework in which the quadrat coordinates are expressed.

  - **Warning**: this function has no obvious use in ecological analyses. It can be used to compile plot-level datasets when coordinates are obtained in the field and are internal to specific quadrats. It should be obsolete and unnecessary if one is using tablets (always with whole-plot-level coordinates) to map the individual trees. In general, this function should not be used often, except during the compilation of the whole dataset. If you find yourself using it in your analytical pipelines, think twice. The general recommendation is to store tree-level location within the whole plot, and not within particular quadrats. All spatial analyses can be derived from outer coordinates, and storing / using inner coordinates are a practical point of vulnerability that should be avoided in general.

These main functions call a few other auxiliary functions:

- two functions to create sequences of letters according to different systems
- `make_quadrats_non_overlapping()`: a possibly useful function that adjusts the limits of a given set of quadrats so they do not overlap with their closest neighbors (k = 8 by default). The function is used internally to `get_quadrat_locations_from_outer_coordinates()`. It does not do anything critical there, and it should not matter for most applications: it is mostly anticipating code extension and unusual cases. If the function breaks there, you should get into the guts of the function and cancel that step.

3 Quadrat ID conventions

3.1 Practical need for generalization

Quadrat-level IDs could be arbitrary, making IDing conventions irrelevant, if teams / projects / networks kept the quadrat-level locations explicitly stored in quadrat-level tables. Handling

quadrats and inner/outer coordinates would be straightforward.

The new code has been developed under the pessimistic scenario that the user does not have a explicit table with the locations of the quadrats. This requires extracting locations from quadrat IDs.

CTFS / `fgeo` code is very concrete when it refers to these conventions: it only accepts four cases (CCRR starting at 0 or 1, CR for 5 x5 m quadrats, and consecutive numerical indexes). Because of this limitation, these functions are not broadly usable outside the most orthodox CTFS tables. A review of ~20 datasets (see evaluation folder) shows that there is substantial variation in the ways different teams (or the same team in different moments, or the same team for different tables) identify quadrats. Often, the conventions are different for quadrats at different resolution.

The new code tries to handle a much broader range of quadrat IDing conventions than CTFS / `fgeo` code. Because extracting location from ID should be a rare step, the code does not take much risk. Instead, it asks for a very detailed description of the quadrat IDing convention. These descriptions are based on a long set of (partially redundant) parameters:

Note that the code gets locations + IDs. It will ask for the basic information of unit and sub-unit size.

3.2 Parameters for `make_quadrats()`

The `make_quadrats()` function accepts complex quadrat ID conventions, through a long list of potential parameters.

- `outer.xmin`, `outer.ymin`: Minimum x and y coordinates of the outer coordinate extent. These values define the lower-left corner of the coordinate frame in which quadrat locations will be expressed. If missing, both are assumed to be `0`.
- `outer.xmax`, `outer.ymax`: Maximum x and y coordinates of the outer coordinate extent. These values define the upper bounds of the coordinate frame in which quadrat locations will be expressed. These parameters are required. If the unit split into sub-units is the entire plot, these parameters are equivalent to `plotdim` parameter in CTFS / `fgeo`.
- `quadrat.side.x`, `quadrat.side.y`: Width and height of the quadrats = sub-units within the outer unit. If only one is supplied, the other is assumed to be equal.
  - If the outer extent is not exactly divisible by these values, quadrats along the outer edge are retained but made smaller. These smaller pieces are preferred over larger-than-usual quadrats because they would imply smaller data losses if the user wants to filter and keep only quadrats of the same size for ecological analyses.

- `built`: Rule used to build quadrat IDs. Use `"col+row"` when the ID is built from the column label followed by the row label, `"row+col"` when the row label comes first, or `"sequence"` when each quadrat receives one continuous sequential label.
- `example.part.for.col`: Prototype of the column part of the quadrat ID. This determines whether column labels are numeric or alphabetic, whether letters are upper- or lower-case, and how much zero-padding is used. Examples:
  - 1 or "1" will generate the sequence 1, 2, 3, 4, …
  - "01" will generate the sequence 01, 02, 03, 04, …
  - 0 or "0" will generate the sequence 0, 1, 2, 3, …, unless `start.col.at` says otherwise.
  - "00" will generate the sequence 00, 01, 02, 03, unless `start.col.at` says otherwise.
  - "00001", "00000001", etc. will use zero padding accordingly.
  - "A" will generate sequence A, B, C, D, …, Z, AA, AB, AC, …
  - "a" will generate sequence a, b, c, d, …, z, aa, ab, ac, …
  - "AA" will generate sequence AA, AB, AC, …
  - "aa" will generate sequence aa, ab, ac, …
- `example.part.for.row`: Same as `example.part.for.col`, but for rows.
- `example.part.for.sequence`: Prototype of the complete quadrat ID when `built = "sequence"`. As above, this determines whether column labels are numeric or alphabetic, whether letters are upper- or lower-case, and how much zero-padding is used.
- `prefix`: Optional string added before every generated quadrat ID. For example, `prefix = "Q"` can produce IDs such as `"Q0101"`.
- `suffix`: Optional string added after every generated quadrat ID. For example, `suffix = "_A"` can produce IDs such as `"0101_A"`.
- `separator`: Optional string inserted between column and row parts when `built = "col+row"` or `built = "row+col"`. Examples: `""`, `"-"`, `"_"`, `", "` can result in IDs like `"0000"`, `"00-00"`, `"00_00"`, `"00, 00"`, etc.
- `seq.of.col.parts.along.one.row`: Optional custom sequence of column labels. If supplied, it replaces the automatically generated column labels. Its length must equal the number of quadrat columns. This can be useful if one has quadrat IDs based on coordinates, like `"00-00"` followed by`"20-00"` followed by `"40-00"`, etc., and other specific cases.

- `seq.of.row.parts.along.one.col`: Same as `seq.of.col.parts.along.one.row`, but for rows.
- `entire.sequence`: Optional complete vector of quadrat IDs. If supplied, it replaces all automatically generated IDs. Its length should match the total number of generated quadrats.
- `start.col.at`: First value used for automatically generated column labels. For numeric labels, this controls whether columns start at `0`, `1`, or another value. For the moment, it does not work for letters (but that can change, check it).
- `start.row.at`: First value used for automatically generated row labels. By default, it follows `start.col.at`.
- `start.sequence.at`: First value used for automatically generated sequential IDs when `built = "sequence"`.
- `along.columns.first.then.next.column`: Logical. Used only when `built = "sequence"`. If `TRUE`, sequential IDs are assigned along one column first, then continue in the next column.
- `along.rows.first.then.next.row`: Logical. Used only when `built = "sequence"`. If `TRUE`, sequential IDs are assigned along one row first, then continue in the next row. Exactly one of `along.columns.first.then.next.column` and `along.rows.first.then.next.row` should be `TRUE`.
- `changing.directions`: Logical. Used only when `built = "sequence"`. If `TRUE`, the sequential labelling direction alternates between consecutive columns or rows, producing a back-and-forth sequence.

If your quadrat IDing convention cannot be fully described with the paramaters above, in one or two passes, my suggestion is to build a quadrat-level table containing quadrat ID and (xmin, ymin, xmax, ymax) of your quadrats **by hand**, and use that directly. Alternatively, you could open an issue in GitHub, and/or modify the code directly, do a pull request, etc.

4 Use cases

There are three blocks of use cases. The first gets the quadrat-level table with {quadrat ID, xmin, ymin, xmax, ymax}. The second uses that table to move between outer and inner coordinates and between ID conventions. The third shows some quadrat-level summaries.

```r
# This is my local path. You may have to adapt this.
g <- regexpr("fgeo2", getwd(), fixed = TRUE)

if(g == -1)
  stop("the fgeo2 project folder was not found")

path <- substr(getwd(), 1, g + attr(g, "match.length"))

# Source the functions.
f <- paste0(path, "03_spatial/R/new/quadrats.R")
source(f)

# Load tree- and quadrat-level data from Luquillo.
load(paste0(path, "data_examples/luquillo.full2.rdata"))
data <- luquillo.full2

luquillo.quadrats <- read.csv(
  paste0(path, "data_examples/luquillo.quadrats.20x20.csv"),
  colClasses = c(
    quadrat_id = "character",
    xmin = "numeric",
    ymin = "numeric",
    xmax = "numeric",
    ymax = "numeric"
  )
)

# Keep leading zeros in the quadrat IDs stored with the trees.
data$quadrat_id <- NA_character_
has.quadrat <- !is.na(data$quadrat)
data$quadrat_id[has.quadrat] <- sprintf(
  "%04d",
  as.integer(data$quadrat[has.quadrat])
)

dim(data)
```

```
[1] 125446     20
```

```r
head(luquillo.quadrats)
```

```
  quadrat_id xmin ymin xmax ymax
```

```
1        0101     0    0    20    20
2        0102     0   20    20    40
3        0103     0   40    20    60
4        0104     0   60    20    80
5        0105     0   80    20   100
6        0106     0  100    20   120
```

4.1 Getting the quadrat-level table

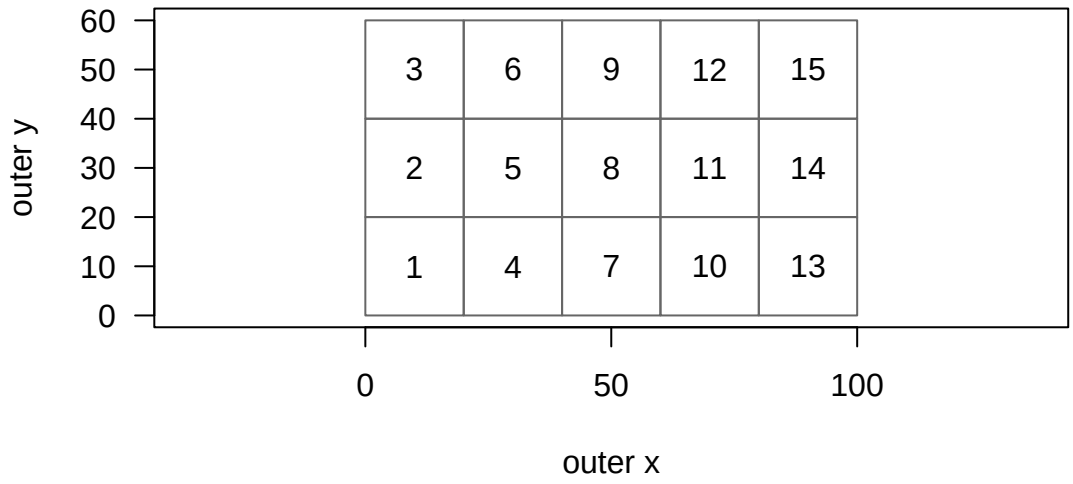
Make simple quadrats from scratch

`make_quadrats_fast()` creates temporary quadrats with simple sequential IDs. Small remainders at the right or top edges are retained.

```
simple.quadrats <- make_quadrats_fast(
  outer.xmin = 0,
  outer.ymin = 0,
  outer.xmax = 100,
  outer.ymax = 60,
  quadrat.side.x = 20,
  quadrat.side.y = 20
)

simple.quadrats
```

```
   quadrat_id xmin ymin xmax ymax
1           1    0    0   20   20
2           2    0   20   20   40
3           3    0   40   20   60
4           4   20    0   40   20
5           5   20   20   40   40
6           6   20   40   40   60
7           7   40    0   60   20
8           8   40   20   60   40
9           9   40   40   60   60
10         10   60    0   80   20
11         11   60   20   80   40
12         12   60   40   80   60
13         13   80    0  100   20
14         14   80   20  100   40
15         15   80   40  100   60
```

```r
plot(
  c(0, 100),
  c(0, 60),
  type = "n",
  asp = 1,
  xlab = "outer x",
  ylab = "outer y",
  las = 1
)

with(
  simple.quadrats,
  rect(xmin, ymin, xmax, ymax, border = "grey40")
)

with(
  simple.quadrats,
  text((xmin + xmax) / 2, (ymin + ymax) / 2, quadrat_id)
)
```



Create quadrats from tree locations and membership

`get_quadrat_locations_from_outer_coordinates()` accepts missing values. Rows with missing quadrat membership, x, or y are not used to infer locations. This is important for the `prior` stems in the Luquillo data.

```r
complete.for.inference <-
  !is.na(data$quadrat_id) &
  !is.na(data$gx) &
  !is.na(data$gy)

sum(complete.for.inference)
```

```
[1] 41576
```

```r
inferred.quadrats <- get_quadrat_locations_from_outer_coordinates(
  quadrat.id = data$quadrat_id,
  outer.x = data$gx,
  outer.y = data$gy,
  outer.xmin = 0,
  outer.ymin = 0,
  outer.xmax = 320,
  outer.ymax = 500,
  regular = TRUE
)

dim(inferred.quadrats)
```

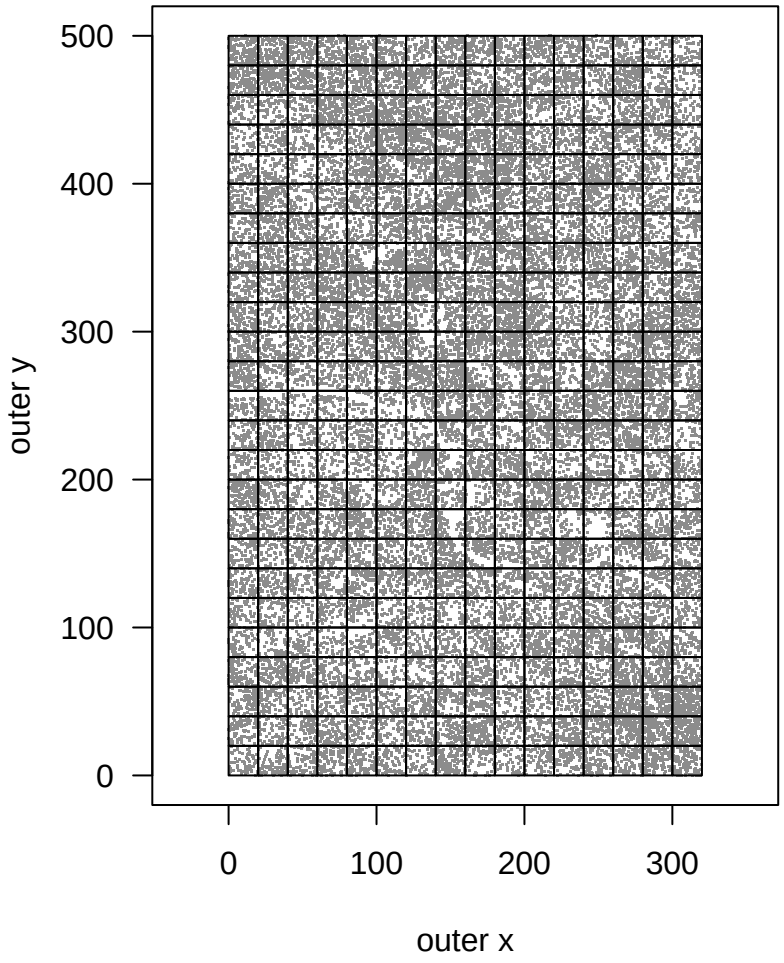
```
[1] 400   5
```

```r
head(inferred.quadrats)
```

```
     quadrat_id xmin ymin xmax ymax
0101       0101    0    0   20   20
0102       0102    0   20   20   40
0103       0103    0   40   20   60
0104       0104    0   60   20   80
0105       0105    0   80   20  100
0106       0106    0  100   20  120
```

```r
plot(
  data$gx[complete.for.inference],
  data$gy[complete.for.inference],
  pch = ".",
  col = "grey55",
```

```
  asp = 1,
  xlab = "outer x",
  ylab = "outer y",
  las = 1
)

with(
  inferred.quadrats,
  rect(xmin, ymin, xmax, ymax, border = "black")
)
```



The output should be checked and stored once, rather than inferred again within later analyses.

Create quadrats from an ID convention: easy cases

Luquillo uses column + row IDs, starting at `01` and without a separator.

```
ccrr.quadrats <- make_quadrats(
  outer.xmin = 0,
  outer.ymin = 0,
  outer.xmax = 320,
  outer.ymax = 500,
  quadrat.side.x = 20,
  quadrat.side.y = 20,
  built = "col+row",
  example.part.for.col = "01",
  example.part.for.row = "01",
  start.col.at = 1,
  start.row.at = 1,
  separator = ""
)

head(ccrr.quadrats)
```

```
     quadrat_id xmin ymin xmax ymax
0101       0101    0    0   20   20
0102       0102    0   20   20   40
0103       0103    0   40   20   60
0104       0104    0   60   20   80
0105       0105    0   80   20  100
0106       0106    0  100   20  120
```

```
tail(ccrr.quadrats)
```

```
     quadrat_id xmin ymin xmax ymax
1620       1620  300  380  320  400
1621       1621  300  400  320  420
1622       1622  300  420  320  440
1623       1623  300  440  320  460
1624       1624  300  460  320  480
1625       1625  300  480  320  500
```

A continuous sequence is simpler. The example below numbers quadrats along one column first and then moves to the next column.

```r
sequence.quadrats <- make_quadrats(
  outer.xmin = 0,
  outer.ymin = 0,
  outer.xmax = 100,
  outer.ymax = 60,
  quadrat.side.x = 20,
  quadrat.side.y = 20,
  built = "sequence",
  example.part.for.sequence = "001",
  start.sequence.at = 1,
  along.columns.first.then.next.column = TRUE,
  along.rows.first.then.next.row = FALSE
)

head(sequence.quadrats)
```

```
    quadrat_id xmin ymin xmax ymax
001        001    0    0   20   20
002        002    0   20   20   40
003        003    0   40   20   60
004        004   20    0   40   20
005        005   20   20   40   40
006        006   20   40   40   60
```

Create quadrats from an ID convention: difficult cases

Some IDs inherit the ID of a larger unit. In Amacayacu, for example, hectares are identified by letters and the 20 x 20 m quadrats within each hectare are numbered `01`, `02`, …, `25`. This requires two steps.

First, obtain the hectare table. Here two adjacent hectares are used only to keep the example small. If hectare locations cannot be inferred safely from their letters, this first table must be supplied explicitly. Then split each hectare separately and add the hectare ID as a prefix.

```r
hectares <- make_quadrats(
  outer.xmin = 0,
  outer.ymin = 0,
  outer.xmax = 200,
  outer.ymax = 100,
  quadrat.side.x = 100,
  quadrat.side.y = 100,
```

```
  built = "sequence",
  example.part.for.sequence = "A",
  start.sequence.at = "A"
)

hectares
```

```
  quadrat_id xmin ymin xmax ymax
A          A    0    0  100  100
B          B  100    0  200  100
```

```
amacayacu.quadrats <- vector("list", nrow(hectares))

for(i in seq_len(nrow(hectares)))
{
  q <- make_quadrats(
    outer.xmin = hectares$xmin[i],
    outer.ymin = hectares$ymin[i],
    outer.xmax = hectares$xmax[i],
    outer.ymax = hectares$ymax[i],
    quadrat.side.x = 20,
    quadrat.side.y = 20,
    built = "sequence",
    example.part.for.sequence = "01",
    start.sequence.at = 1,
    prefix = hectares$quadrat_id[i]
  )

  q$hectare_id <- hectares$quadrat_id[i]
  q <- q[, c(
    "quadrat_id", "hectare_id",
    "xmin", "ymin", "xmax", "ymax"
  )]

  amacayacu.quadrats[[i]] <- q
}

amacayacu.quadrats <- do.call(rbind, amacayacu.quadrats)
rownames(amacayacu.quadrats) <- NULL

head(amacayacu.quadrats)
```

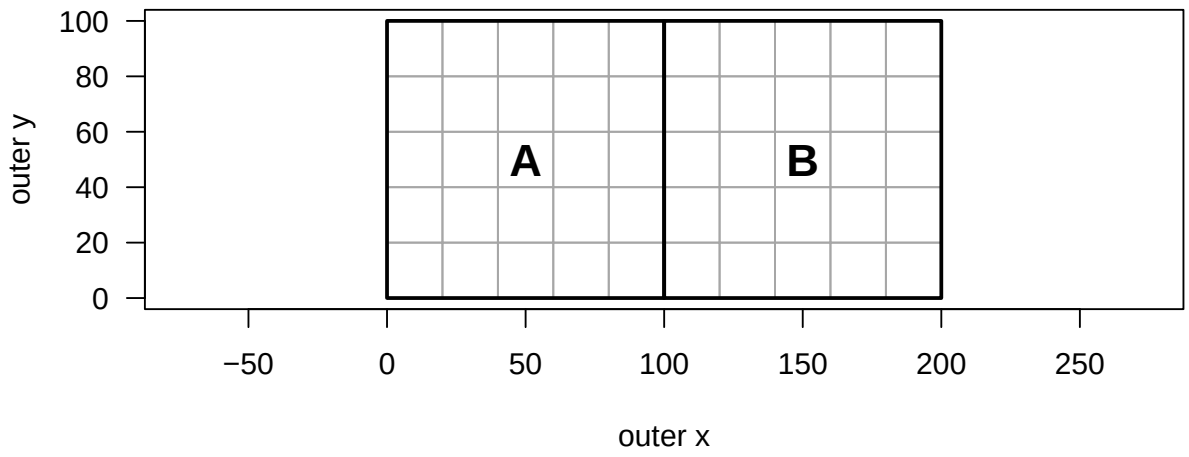
```
  quadrat_id hectare_id xmin ymin xmax ymax
1        A01          A    0    0   20   20
2        A02          A    0   20   20   40
3        A03          A    0   40   20   60
4        A04          A    0   60   20   80
5        A05          A    0   80   20  100
6        A06          A   20    0   40   20
```

```
tail(amacayacu.quadrats)
```

```
   quadrat_id hectare_id xmin ymin xmax ymax
45        B20          B  160   80  180  100
46        B21          B  180    0  200   20
47        B22          B  180   20  200   40
48        B23          B  180   40  200   60
49        B24          B  180   60  200   80
50        B25          B  180   80  200  100
```

```
plot(
  c(0, 200),
  c(0, 100),
  type = "n",
  asp = 1,
  xlab = "outer x",
  ylab = "outer y",
  las = 1
)

with(
  amacayacu.quadrats,
  rect(xmin, ymin, xmax, ymax, border = "grey65")
)

with(
  hectares,
  rect(xmin, ymin, xmax, ymax, border = "black", lwd = 2)
)

with(
  hectares,
  text(
```

```
    (xmin + xmax) / 2,
    (ymin + ymax) / 2,
    quadrat_id,
    cex = 1.5,
    font = 2
  )
)
```



The order of `01` to `25` must be adapted if the field convention runs in another direction. For arbitrary or irregular arrangements, construct the same five-column table directly from the project maps or field records.

Double-checking

The inferred table, the convention-derived table, and the stored table agree for Luquillo.

```
# Recover quadrat locations from the trees.
inferred.quadrats <- get_quadrat_locations_from_outer_coordinates(
  quadrat.id = data$quadrat_id,
  outer.x = data$gx,
  outer.y = data$gy,
  outer.xmin = 0,
  outer.ymin = 0,
  outer.xmax = 320,
  outer.ymax = 500,
  regular = TRUE
)
```

```
# Recover the same locations from the ID convention.
ccrr.quadrats <- make_quadrats(
  outer.xmin = 0,
  outer.ymin = 0,
  outer.xmax = 320,
  outer.ymax = 500,
  quadrat.side.x = 20,
  quadrat.side.y = 20,
  built = "col+row",
  example.part.for.col = "01",
  example.part.for.row = "01",
  start.col.at = 1,
  start.row.at = 1,
  separator = ""
)

rownames(inferred.quadrats) <- NULL
rownames(ccrr.quadrats) <- NULL

identical(inferred.quadrats, luquillo.quadrats)
```

```
[1] TRUE
```

```
identical(ccrr.quadrats, luquillo.quadrats)
```

```
[1] TRUE
```

Basic checks should include IDs, dimensions, and the represented extent.

```
checks <- c(
  no_missing_values = !anyNA(luquillo.quadrats),
  unique_ids = !anyDuplicated(luquillo.quadrats$quadrat_id),
  positive_widths = all(luquillo.quadrats$xmax > luquillo.quadrats$xmin),
  positive_heights = all(luquillo.quadrats$ymax > luquillo.quadrats$ymin),
  correct_extent = identical(
    c(
      min(luquillo.quadrats$xmin),
      min(luquillo.quadrats$ymin),
      max(luquillo.quadrats$xmax),
      max(luquillo.quadrats$ymax)
```

```
    ),
    c(0, 0, 320, 500)
  ),
  expected_total_area = sum(
    (luquillo.quadrats$xmax - luquillo.quadrats$xmin) *
    (luquillo.quadrats$ymax - luquillo.quadrats$ymin)
  ) == 320 * 500
)

checks
```

```
  no_missing_values          unique_ids     positive_widths    positive_heights
               TRUE                 TRUE                TRUE                TRUE
     correct_extent expected_total_area
               TRUE                TRUE
```

Do not automatically swap or replace bad limits. Check them against the original map or another trusted source. Here a deliberately damaged row is restored from the stored Luquillo table.

```
quadrats.problem <- luquillo.quadrats[1:3, ]
quadrats.problem$xmax[2] <- quadrats.problem$xmin[2]

bad <-
  quadrats.problem$xmax <= quadrats.problem$xmin |
  quadrats.problem$ymax <= quadrats.problem$ymin

quadrats.problem[bad, ]
```

```
  quadrat_id xmin ymin xmax ymax
2       0102    0   20    0   40
```

```
i <- match(
  quadrats.problem$quadrat_id[bad],
  luquillo.quadrats$quadrat_id
)

quadrats.problem[bad, c("xmin", "ymin", "xmax", "ymax")] <-
  luquillo.quadrats[i, c("xmin", "ymin", "xmax", "ymax")]
```

```
stopifnot(
  all(quadrats.problem$xmax > quadrats.problem$xmin),
  all(quadrats.problem$ymax > quadrats.problem$ymin)
)
```

The final check is to recalculate tree membership from coordinates. The Luquillo table uses the `left_bottom` boundary rule: left and bottom limits are included; right and top limits are excluded, except at the outer plot boundary.

```
check.i <- which(
  !is.na(data$quadrat_id) &
  !is.na(data$gx) &
  !is.na(data$gy)
)

checked.membership <- assign_points_to_quadrats(
  outer.x = data$gx[check.i],
  outer.y = data$gy[check.i],
  point.id = as.character(check.i),
  quadrat.locations = luquillo.quadrats,
  boundary.rule = "left_bottom"
)

all(checked.membership$quadrat_id == data$quadrat_id[check.i])
```

```
[1] TRUE
```

Overlapping quadrats produce more than one row for the affected points. This warning should not be ignored unless the overlap is intentional.

```
overlapping.quadrats <- luquillo.quadrats[
  match(c("0101", "0201"), luquillo.quadrats$quadrat_id),
]
overlapping.quadrats$xmin[2] <- 10

assign_points_to_quadrats(
  outer.x = 15,
  outer.y = 10,
  point.id = "example point",
  quadrat.locations = overlapping.quadrats
)
```

```
Warning in assign_points_to_quadrats(outer.x = 15, outer.y = 10, point.id =
"example point", : 1 point(s) were assigned to more than one quadrat; this is
expected if quadrats overlap
```

```
       point_id outer.x outer.y quadrat_id inner.x inner.y
1 example point      15      10       0101      15      10
2 example point      15      10       0201       5      10
```

4.2 Using a quadrat-level table

Get inner coordinates from outer coordinates

The example combines six mapped trees and one record with missing coordinates. Missing and unassigned points are retained with `NA` membership and inner coordinates.

```
mapped.i <- which(!is.na(data$gx) & !is.na(data$gy))[1:6]
missing.i <- which(is.na(data$gx) | is.na(data$gy))[1]
point.i <- c(mapped.i, missing.i)

example.points <- data[point.i, c("treeID", "gx", "gy")]

assigned.points <- assign_points_to_quadrats(
  outer.x = example.points$gx,
  outer.y = example.points$gy,
  point.id = example.points$treeID,
  quadrat.locations = luquillo.quadrats,
  boundary.rule = "left_bottom"
)
```

```
Warning in assign_points_to_quadrats(outer.x = example.points$gx, outer.y =
example.points$gy, : 1 point(s) were not assigned to any quadrat; quadrat_id,
inner.x, and inner.y were set to NA
```

```
assigned.points
```

```
  point_id outer.x outer.y quadrat_id inner.x inner.y
1       19  305.93  216.96       1611    5.93   16.96
2       20   91.26  167.30       0509   11.26    7.30
3       42    0.60    2.23       0101    0.60    2.23
4       43   57.27    8.51       0301   17.27    8.51
```

```
5        44    0.97   22.15        0102    0.97    2.15
6        46  164.35  415.93        0921    4.35   15.93
7         1      NA      NA        <NA>      NA      NA
```

Get outer coordinates from inner coordinates

`compile_points_from_quadrats()` reverses the previous calculation. Rows without a quadrat assignment are omitted here because there is no location to reconstruct.

```
# Recreate the preceding assignment so this example can run by itself.
mapped.i <- which(!is.na(data$gx) & !is.na(data$gy))[1:6]

assigned.points <- assign_points_to_quadrats(
  outer.x = data$gx[mapped.i],
  outer.y = data$gy[mapped.i],
  point.id = data$treeID[mapped.i],
  quadrat.locations = luquillo.quadrats,
  boundary.rule = "left_bottom"
)

compiled.points <- compile_points_from_quadrats(
  inner.x = assigned.points$inner.x,
  inner.y = assigned.points$inner.y,
  point.id = assigned.points$point_id,
  quadrat.id = assigned.points$quadrat_id,
  quadrat.locations = luquillo.quadrats
)

original.points <- assigned.points[, c(
  "point_id", "outer.x", "outer.y"
)]

compiled.points
```

```
  point_id outer.x outer.y
1       19  305.93  216.96
2       20   91.26  167.30
3       42    0.60    2.23
4       43   57.27    8.51
5       44    0.97   22.15
6       46  164.35  415.93
```

```r
all.equal(compiled.points, original.points, check.attributes = FALSE)
```

```
[1] TRUE
```

Use two or more quadrat ID conventions

The same grid can be labelled with row + column IDs. Match explicit locations to construct the crosswalk; do not translate the strings directly.

```r
rrcc.quadrats <- make_quadrats(
  outer.xmin = 0,
  outer.ymin = 0,
  outer.xmax = 320,
  outer.ymax = 500,
  quadrat.side.x = 20,
  quadrat.side.y = 20,
  built = "row+col",
  example.part.for.col = "01",
  example.part.for.row = "01",
  start.col.at = 1,
  start.row.at = 1,
  separator = ""
)

id.crosswalk <- merge(
  luquillo.quadrats,
  rrcc.quadrats,
  by = c("xmin", "ymin", "xmax", "ymax"),
  all = TRUE,
  suffixes = c("_CCRR", "_RRCC"),
  sort = FALSE
)

id.crosswalk <- id.crosswalk[, c(
  "quadrat_id_CCRR", "quadrat_id_RRCC",
  "xmin", "ymin", "xmax", "ymax"
)]

stopifnot(!anyNA(id.crosswalk))
head(id.crosswalk)
```

```
  quadrat_id_CCRR quadrat_id_RRCC xmin ymin xmax ymax
1            0101            0101    0    0   20   20
2            0102            0201    0   20   20   40
3            0103            0301    0   40   20   60
4            0104            0401    0   60   20   80
5            0105            0501    0   80   20  100
6            0106            0601    0  100   20  120
```

Nested quadrats within other quadrats

This example splits every 20 x 20 m Luquillo quadrat into sixteen 5 x 5 m quadrats. The smaller IDs inherit the parent ID, for example `0101-11`.

```r
subquadrats <- vector("list", nrow(luquillo.quadrats))

for(i in seq_len(nrow(luquillo.quadrats)))
{
  parent <- luquillo.quadrats[i, ]

  q <- make_quadrats(
    outer.xmin = parent$xmin,
    outer.ymin = parent$ymin,
    outer.xmax = parent$xmax,
    outer.ymax = parent$ymax,
    quadrat.side.x = 5,
    quadrat.side.y = 5,
    built = "col+row",
    example.part.for.col = "1",
    example.part.for.row = "1",
    start.col.at = 1,
    start.row.at = 1,
    prefix = paste0(parent$quadrat_id, "-")
  )

  q$parent_quadrat_id <- parent$quadrat_id
  q <- q[, c(
    "quadrat_id", "parent_quadrat_id",
    "xmin", "ymin", "xmax", "ymax"
  )]

  subquadrats[[i]] <- q
}
```

```r
subquadrats <- do.call(rbind, subquadrats)
rownames(subquadrats) <- NULL

dim(subquadrats)
```

```
[1] 6400    6
```

```r
head(subquadrats)
```

```
  quadrat_id parent_quadrat_id xmin ymin xmax ymax
1    0101-11              0101    0    0    5    5
2    0101-12              0101    0    5    5   10
3    0101-13              0101    0   10    5   15
4    0101-14              0101    0   15    5   20
5    0101-21              0101    5    0   10    5
6    0101-22              0101    5    5   10   10
```

4.3 Quadrat-level ecological tasks

Species richness at different scales

The same trees can be summarized within a regular grid or within randomly located quadrats. The random quadrats below have fixed sizes but independent locations, so they can overlap. Fifty are sampled at each scale.

```r
# Prepare the point data used in this example.
living.i <- which(
  data$status == "A" &
  !is.na(data$gx) &
  !is.na(data$gy) &
  !is.na(data$sp)
)

trees <- data[living.i, c("sp", "gx", "gy")]

# Count species in any supplied quadrat table.
get_richness <- function(quadrats, trees)
{
  membership <- assign_points_to_quadrats(
```

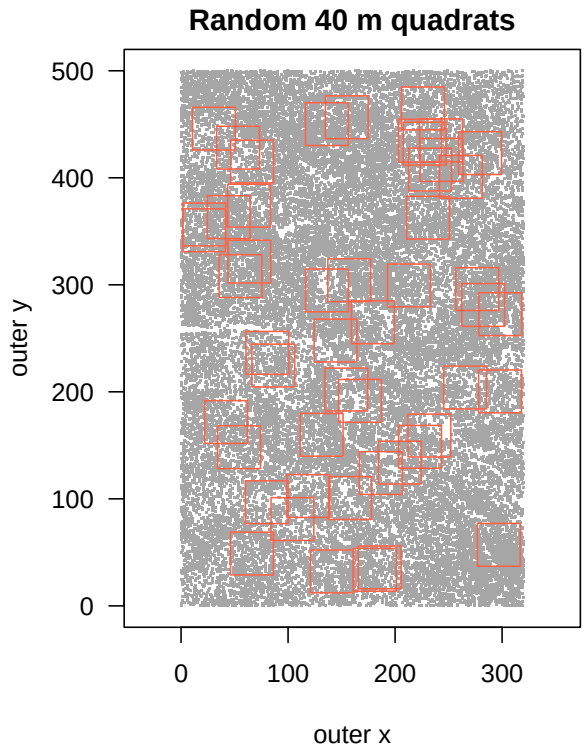
```
    outer.x = trees$gx,
    outer.y = trees$gy,
    point.id = seq_len(nrow(trees)),
    quadrat.locations = quadrats
  )

  membership <- membership[!is.na(membership$quadrat_id), ]

  species.quadrat <- data.frame(
    quadrat_id = membership$quadrat_id,
    sp = trees$sp[as.integer(membership$point_id)]
  )
  species.quadrat <- unique(species.quadrat)

  nsp <- table(species.quadrat$quadrat_id)
  richness <- integer(nrow(quadrats))
  names(richness) <- quadrats$quadrat_id
  richness[names(nsp)] <- as.integer(nsp)

  richness
}

set.seed(87283)
scales <- c(10, 20, 40)
n.random <- 50
richness <- list()

for(i in seq_along(scales))
{
  side.i <- scales[i]

  regular.quadrats <- make_quadrats_fast(
    outer.xmin = 0,
    outer.ymin = 0,
    outer.xmax = 320,
    outer.ymax = 500,
    quadrat.side.x = side.i,
    quadrat.side.y = side.i
  )

  complete.size <-
    regular.quadrats$xmax - regular.quadrats$xmin == side.i &
```

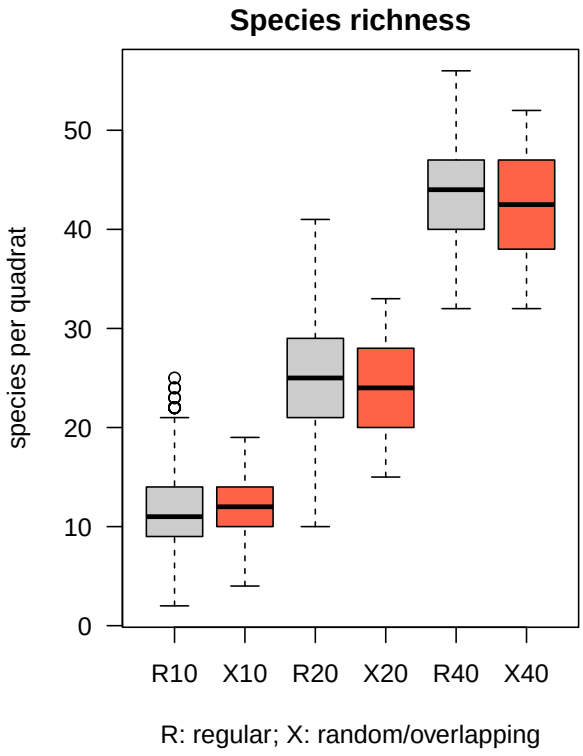
```
    regular.quadrats$ymax - regular.quadrats$ymin == side.i

  regular.richness <- get_richness(regular.quadrats, trees)

  random.quadrats <- data.frame(
    quadrat_id = paste0("random_", side.i, "_", seq_len(n.random)),
    xmin = runif(n.random, 0, 320 - side.i),
    ymin = runif(n.random, 0, 500 - side.i)
  )
  random.quadrats$xmax <- random.quadrats$xmin + side.i
  random.quadrats$ymax <- random.quadrats$ymin + side.i

  richness[[paste0("regular ", side.i, " m")]] <-
    regular.richness[complete.size]

  # Overlap and incomplete coverage are intentional here.
  richness[[paste0("random ", side.i, " m")]] <- suppressWarnings(
    get_richness(random.quadrats, trees)
  )

  if(side.i == max(scales))
    random.to.plot <- random.quadrats
}

round(sapply(richness, mean), 1)
```

```
regular 10 m  random 10 m regular 20 m  random 20 m regular 40 m  random 40 m
        11.5         11.9         25.1         24.3         43.6         42.5
```

```
# Plot one set of random quadrats and the richness distributions.
old.par <- par(no.readonly = TRUE)
par(mfrow = c(1, 2), mar = c(4, 4, 2, 1))

plot(
  trees$gx,
  trees$gy,
  pch = ".",
  col = "grey65",
  asp = 1,
  xlab = "outer x",
  ylab = "outer y",
```

```
  main = "Random 40 m quadrats",
  las = 1
)

with(
  random.to.plot,
  rect(xmin, ymin, xmax, ymax, border = "tomato")
)

boxplot(
  richness,
  names = c("R10", "X10", "R20", "X20", "R40", "X40"),
  col = rep(c("grey80", "tomato"), 3),
  xlab = "R: regular; X: random/overlapping",
  ylab = "species per quadrat",
  main = "Species richness",
  las = 1
)
```





```
par(old.par)
```

Incomplete quadrats along the outer plot edge are removed from the regular-grid summaries. The randomly located quadrats always retain their complete nominal size.

Distances and neighboring quadrats

The distance matrix below uses quadrat centroids. For 400 quadrats it is small enough to calculate directly with base R. The same chunk gets the four nearest quadrats and identifies those sharing an edge.

```
centroids <- cbind(
  x = (luquillo.quadrats$xmin + luquillo.quadrats$xmax) / 2,
  y = (luquillo.quadrats$ymin + luquillo.quadrats$ymax) / 2
)
rownames(centroids) <- luquillo.quadrats$quadrat_id

quadrat.distances <- as.matrix(dist(centroids))
quadrat.distances[1:5, 1:5]
```

```
     0101 0102 0103 0104 0105
0101    0   20   40   60   80
0102   20    0   20   40   60
0103   40   20    0   20   40
0104   60   40   20    0   20
0105   80   60   40   20    0
```

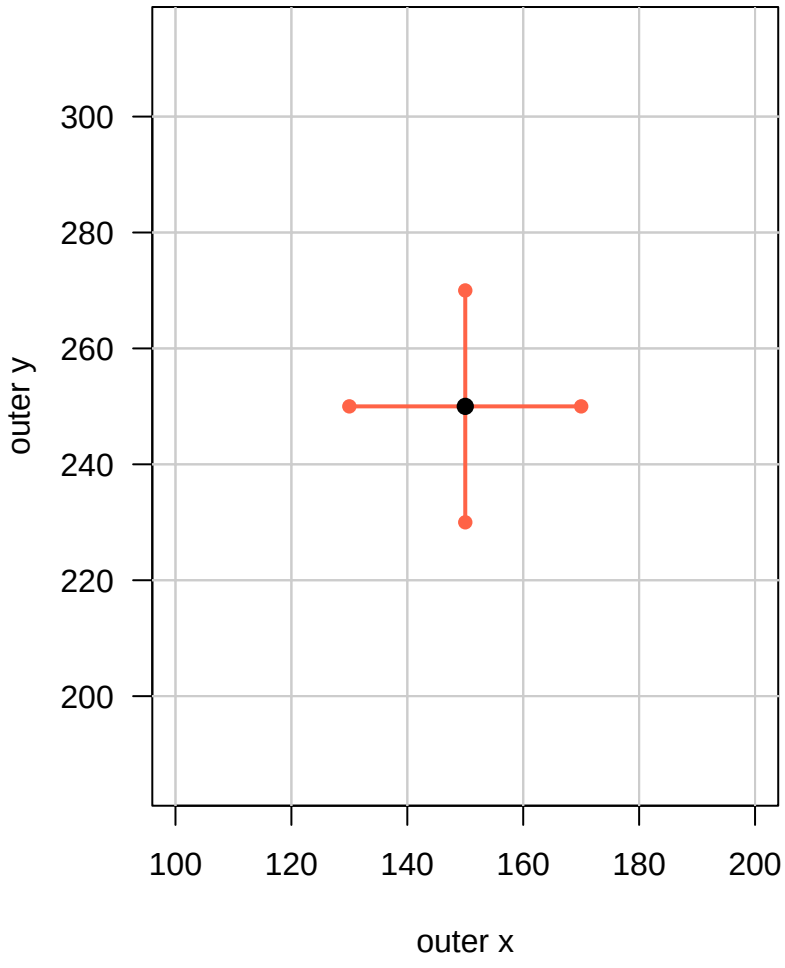
```
diag(quadrat.distances) <- Inf
k <- 4
nearest <- vector("list", nrow(quadrat.distances))

for(i in seq_len(nrow(quadrat.distances)))
{
  j <- order(quadrat.distances[i, ])[seq_len(k)]

  nearest[[i]] <- data.frame(
    quadrat_id = rownames(quadrat.distances)[i],
    neighbor_id = colnames(quadrat.distances)[j],
    distance = quadrat.distances[i, j]
  )
```

```
}

nearest <- do.call(rbind, nearest)
rownames(nearest) <- NULL
shared.edge <- nearest[nearest$distance == 20, ]

head(shared.edge)
```

```
  quadrat_id neighbor_id distance
1       0101        0102       20
2       0101        0201       20
5       0102        0101       20
6       0102        0103       20
7       0102        0202       20
9       0103        0102       20
```

```
# Show the shared-edge neighbors of one central quadrat.
focus.quadrat <- "0813"
focus.neighbors <- shared.edge$neighbor_id[
  shared.edge$quadrat_id == focus.quadrat
]

i <- match(focus.quadrat, rownames(centroids))
j <- match(focus.neighbors, rownames(centroids))

xlim <- range(centroids[c(i, j), 1]) + c(-30, 30)
ylim <- range(centroids[c(i, j), 2]) + c(-30, 30)

plot(
  xlim,
  ylim,
  type = "n",
  asp = 1,
  xlab = "outer x",
  ylab = "outer y",
  las = 1
)

with(
  luquillo.quadrats,
  rect(xmin, ymin, xmax, ymax, border = "grey80")
```

```
)

segments(
  centroids[i, 1],
  centroids[i, 2],
  centroids[j, 1],
  centroids[j, 2],
  col = "tomato",
  lwd = 2
)

points(centroids[i, 1], centroids[i, 2], pch = 16, cex = 1.2)
points(centroids[j, 1], centroids[j, 2], pch = 16, col = "tomato")
```



Edge quadrats still receive four nearest neighbors, but have only two or three shared-edge

neighbors. For much larger quadrat tables, use `FNN::get.knn()` on the centroid coordinates instead of constructing the complete distance matrix.

Abundance, presence, and basal area

Keep a sparse abundance table for ordinary grouped operations. A quadrat x species matrix is convenient when zeros or community comparisons are needed. The `dbh` values are in millimetres; missing diameters are omitted from the basal-area summary.

```
# Prepare the tree data used in this subsection.
living.i <- which(
  data$status == "A" &
  !is.na(data$quadrat_id) &
  !is.na(data$sp)
)

trees <- data[living.i, c("quadrat_id", "treeID", "sp", "dbh")]

abundance <- aggregate(
  treeID ~ quadrat_id + sp,
  data = trees,
  FUN = length
)
names(abundance)[3] <- "abundance"

trees$quadrat_id <- factor(
  trees$quadrat_id,
  levels = luquillo.quadrats$quadrat_id
)
trees$sp <- factor(trees$sp, levels = sort(unique(trees$sp)))

abundance.matrix <- xtabs(~ quadrat_id + sp, data = trees)
presence.matrix <- abundance.matrix > 0

trees$basal_area <- pi * (trees$dbh / 2000)^2
has.dbh <- !is.na(trees$dbh)

basal.area <- aggregate(
  basal_area ~ quadrat_id + sp,
  data = trees[has.dbh, ],
  FUN = sum
)
```

```r
head(abundance)
```

```
  quadrat_id     sp abundance
1       0102 ALCFLO         1
2       0107 ALCFLO         1
3       0113 ALCFLO         1
4       0114 ALCFLO         1
5       0119 ALCFLO         1
6       0124 ALCFLO         1
```

```r
dim(abundance.matrix)
```

```
[1] 400 148
```

```r
head(basal.area)
```

```
  quadrat_id     sp   basal_area
1       0102 ALCFLO 0.0706858347
2       0107 ALCFLO 0.0411870651
3       0113 ALCFLO 0.0298647652
4       0114 ALCFLO 0.0002865211
5       0119 ALCFLO 0.0002138246
6       0124 ALCFLO 0.0002061199
```

The same grouped sum can be applied to a biomass column if biomass has already been estimated for each tree.

Compare communities between quadrats

For two quadrats, Bray-Curtis dissimilarity can be calculated directly from their abundance vectors. This subsection rebuilds the required community matrix from the original data.

```r
living.i <- which(
  data$status == "A" &
  !is.na(data$quadrat_id) &
  !is.na(data$sp)
)
```

```r
community.data <- data[living.i, c("quadrat_id", "sp")]
community.data$quadrat_id <- factor(
  community.data$quadrat_id,
  levels = luquillo.quadrats$quadrat_id
)
community.data$sp <- factor(
  community.data$sp,
  levels = sort(unique(community.data$sp))
)

abundance.matrix <- xtabs(
  ~ quadrat_id + sp,
  data = community.data
)

community.1 <- abundance.matrix["0101", ]
community.2 <- abundance.matrix["0102", ]

bray.curtis <- sum(abs(community.1 - community.2)) /
  sum(community.1 + community.2)

bray.curtis
```

```
[1] 0.4716981
```

For all quadrat pairs, `vegan::vegdist(abundance.matrix, method = "bray")` returns the corresponding distance object.

Summarize neighboring communities

This replaces narrow routines such as `findneighborabund()` and `neighbors()`. The required community matrix and shared-edge pairs are rebuilt here, so the example can run independently.

```r
# Species abundance per quadrat.
living.i <- which(
  data$status == "A" &
  !is.na(data$quadrat_id) &
  !is.na(data$sp)
)
```

```
community.data <- data[living.i, c("quadrat_id", "sp")]
community.data$quadrat_id <- factor(
  community.data$quadrat_id,
  levels = luquillo.quadrats$quadrat_id
)
community.data$sp <- factor(
  community.data$sp,
  levels = sort(unique(community.data$sp))
)

abundance.matrix <- xtabs(
  ~ quadrat_id + sp,
  data = community.data
)
presence.matrix <- abundance.matrix > 0

# Pairs of quadrats that share an edge.
centroids <- cbind(
  x = (luquillo.quadrats$xmin + luquillo.quadrats$xmax) / 2,
  y = (luquillo.quadrats$ymin + luquillo.quadrats$ymax) / 2
)
rownames(centroids) <- luquillo.quadrats$quadrat_id

quadrat.distances <- as.matrix(dist(centroids))
ij <- which(quadrat.distances == 20, arr.ind = TRUE)

shared.edge <- data.frame(
  quadrat_id = rownames(quadrat.distances)[ij[, 1]],
  neighbor_id = colnames(quadrat.distances)[ij[, 2]]
)

# Summarize one species among neighboring quadrats.
focus.sp <- names(which.max(colSums(abundance.matrix)))
focus.abundance <- abundance.matrix[, focus.sp]
focus.presence <- presence.matrix[, focus.sp]

shared.edge$abundance <- focus.abundance[shared.edge$neighbor_id]
shared.edge$presence <- focus.presence[shared.edge$neighbor_id]

neighbor.summary <- aggregate(
  cbind(abundance, presence) ~ quadrat_id,
  data = shared.edge,
```

```
  FUN = sum
)

n.neighbors <- table(shared.edge$quadrat_id)
neighbor.summary$proportion_present <-
  neighbor.summary$presence /
  as.integer(n.neighbors[neighbor.summary$quadrat_id])

focus.sp
```

```
[1] "PALRIP"
```

```
head(neighbor.summary)
```

```
  quadrat_id abundance presence proportion_present
1       0101        24        2          1.0000000
2       0102        58        3          1.0000000
3       0103       127        3          1.0000000
4       0104        96        3          1.0000000
5       0105        53        3          1.0000000
6       0106        21        2          0.6666667
```